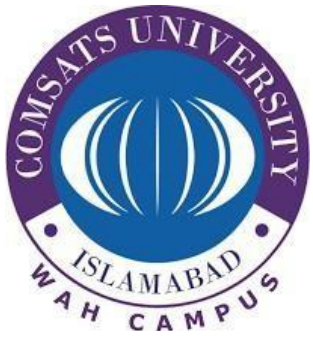


DEVOPS FOR CLOUD COMPUTING

Assignment

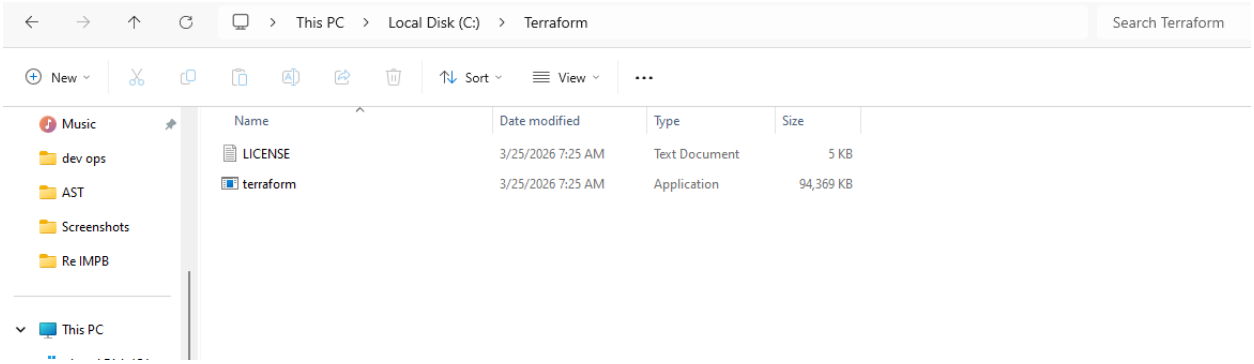
Name:
Tehreem Mirza

Registeration No:
FA22-BSE-088

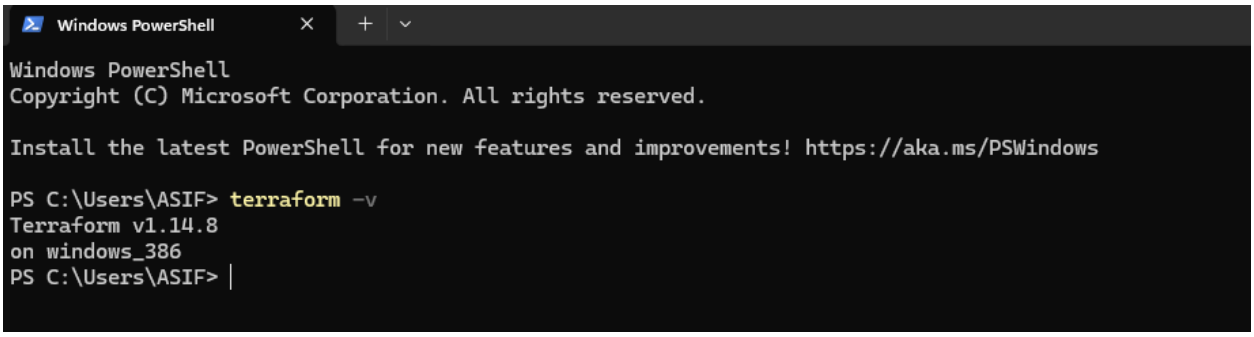
Section:
BSE-8B

Submitted To:
Sir Muhammad Ibrahim

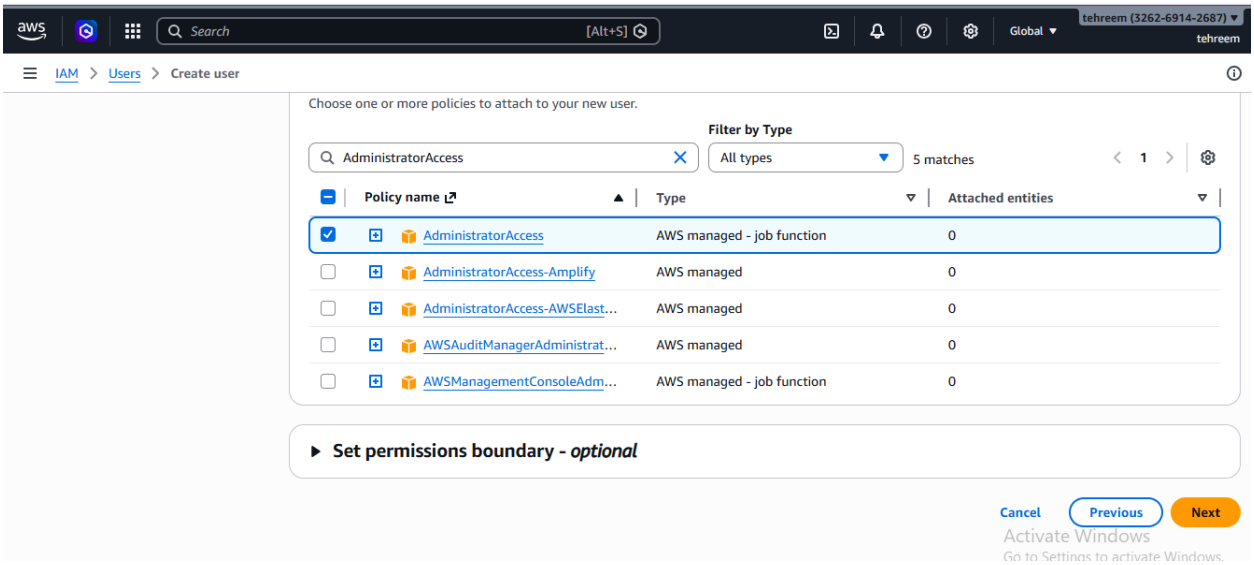
Installed terraform on windows and also pasted path on environmental variables



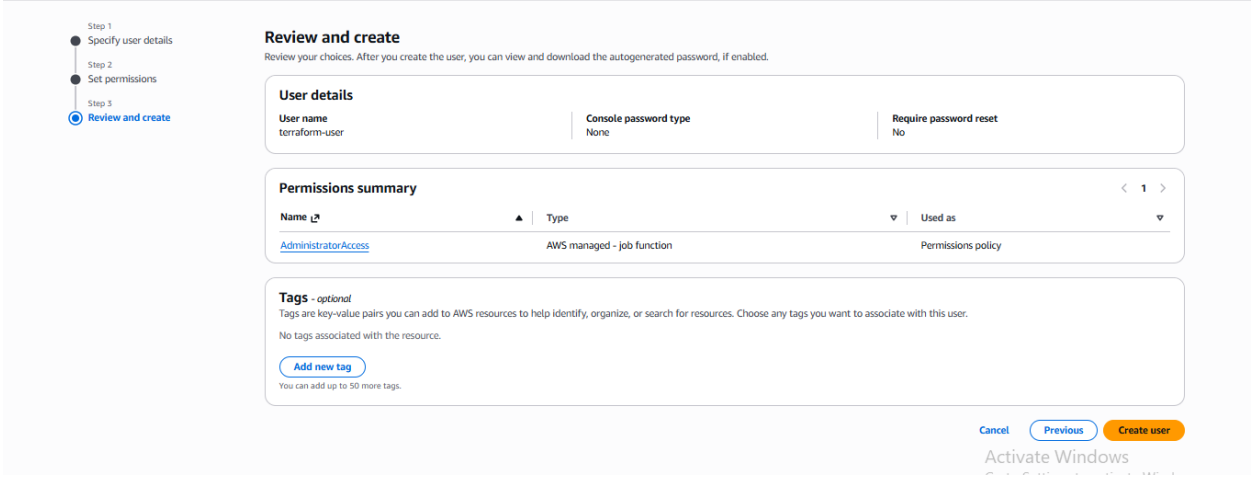
Checking if terraform is installed on windows

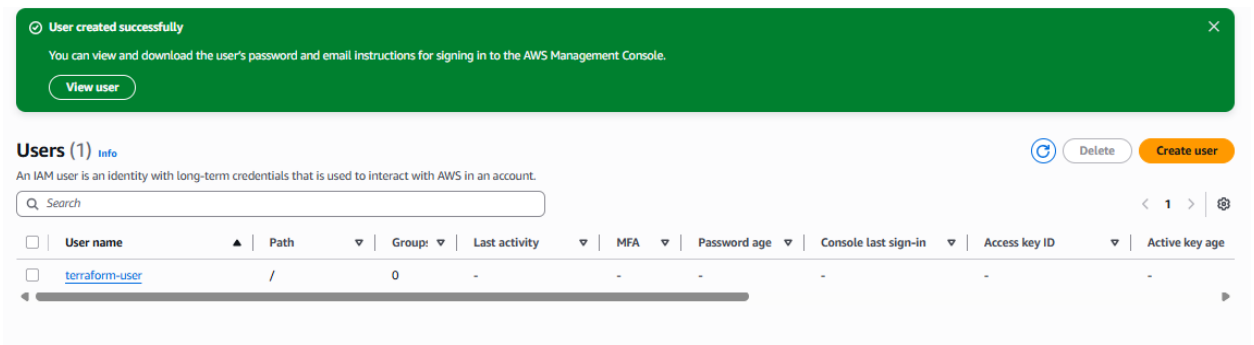


Configuring AWS cli on my PC ---Searched IAM then creating user on AWS

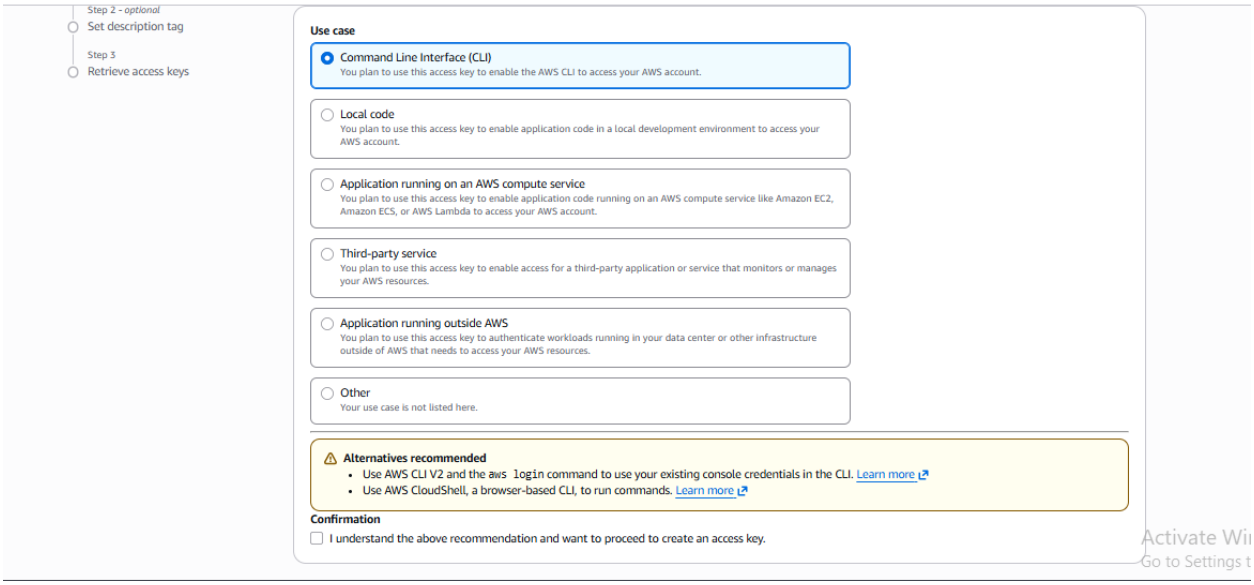


User has been created





After clicking on the terraform user now creating access keys




```
PS C:\Users\ASIF> winget install Amazon.AWSCLI
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function pro
rly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes  [N] No: yes
Found AWS Command Line Interface [Amazon.AWSCLI] Version 2.34.19
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://awscli.amazonaws.com/AWSCLIV2-2.34.19.msi
  45.1 MB / 45.1 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\Users\ASIF>
```

```
PS C:\Users\ASIF> aws --version
aws-cli/2.34.19 Python/3.14.3 Windows/11 exe/AMD64
PS C:\Users\ASIF>
```

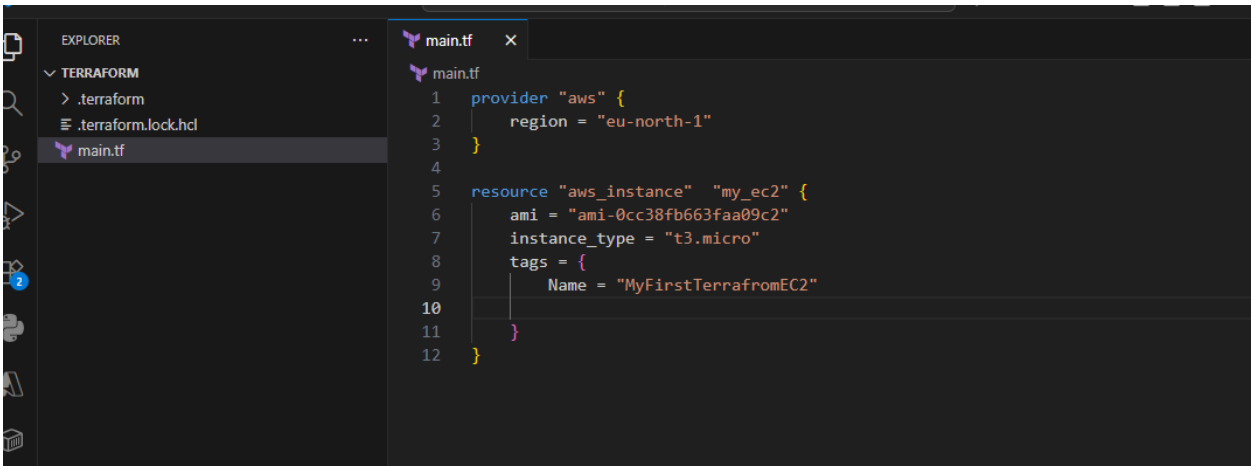
```
PS C:\Users\ASIF> aws --version
aws-cli/2.34.19 Python/3.14.3 Windows/11 exe/AMD64
PS C:\Users\ASIF> aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws log
in'.

AWS Access Key ID [None]: AKIAUX5ZIA2PQXMEXRCA
AWS Secret Access Key [None]: QbwMyyPgGfuwiVQgsjyVKM9uk+vXeOxjKfa4nifG
Default region name [None]: eu-north-1
Default output format [None]: json
PS C:\Users\ASIF>
```

Creating first AWS EC2 instance:

Created main.tf file:

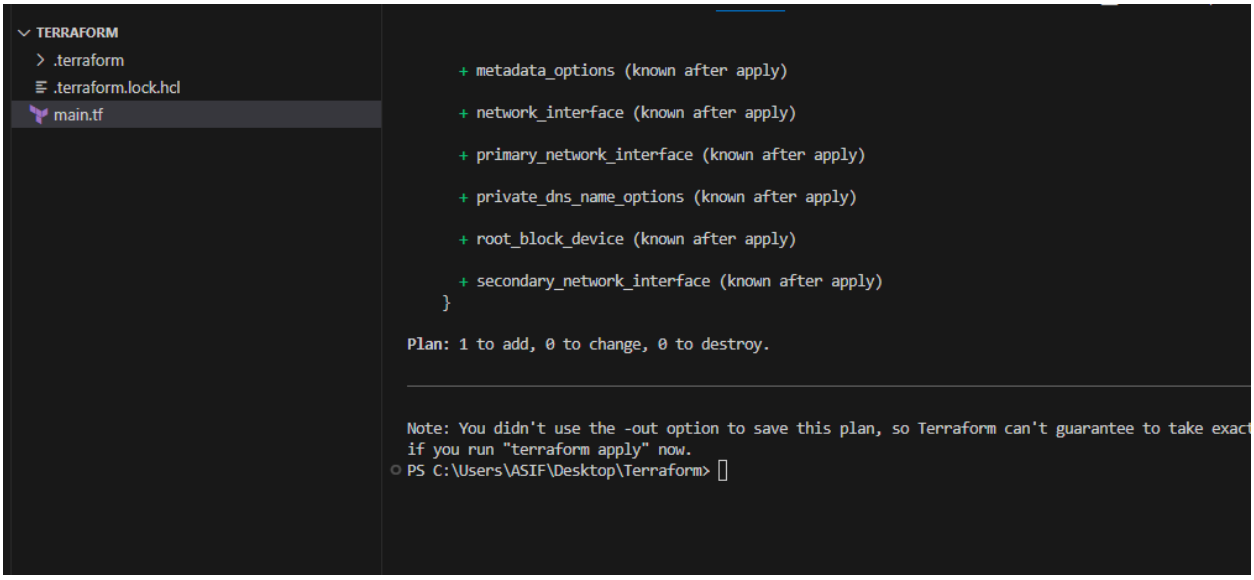


```
PS C:\Users\ASIF\Desktop\Terraform> terraform plan

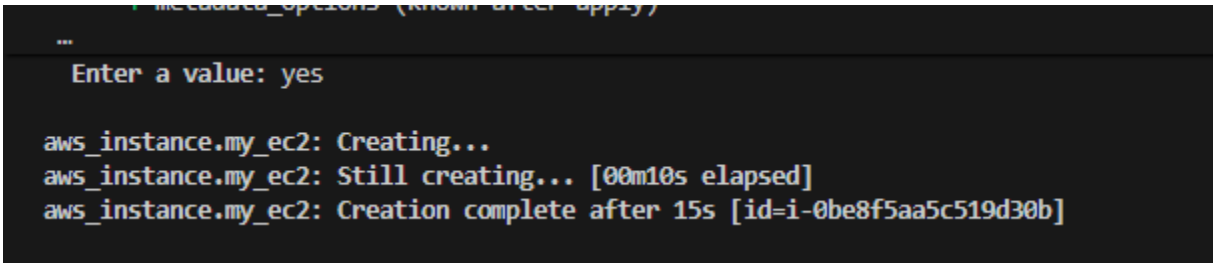
Terraform used the selected providers to generate the following execution plan. Resource a
the following symbols:
  + create

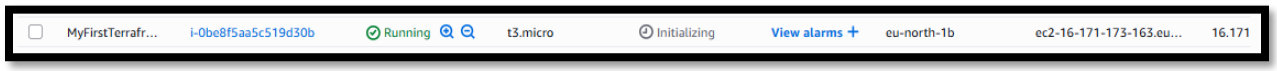
Terraform will perform the following actions:

  # aws_instance.my_ec2 will be created
  + resource "aws_instance" "my_ec2" {
      + ami                                  = "ami-0cc38fb663faa09c2"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
```

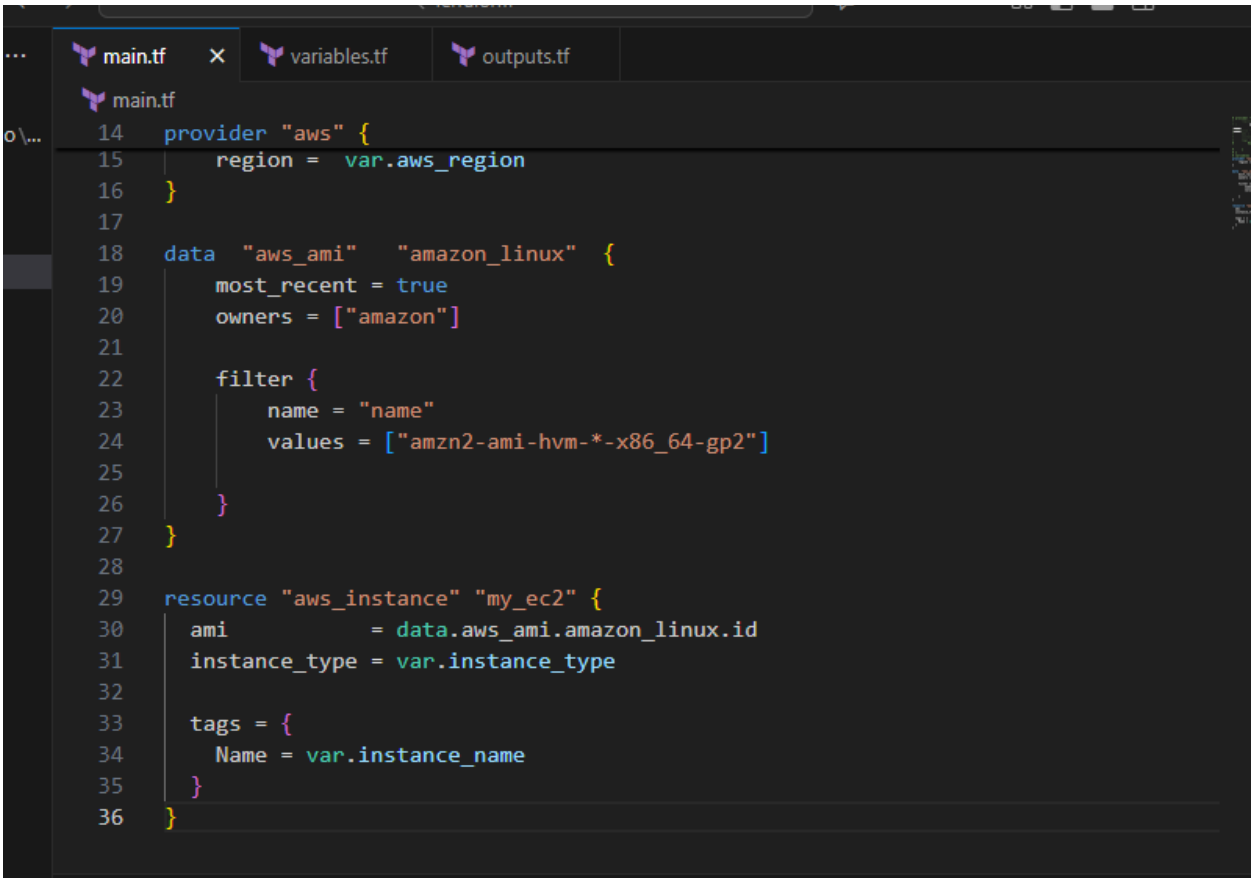


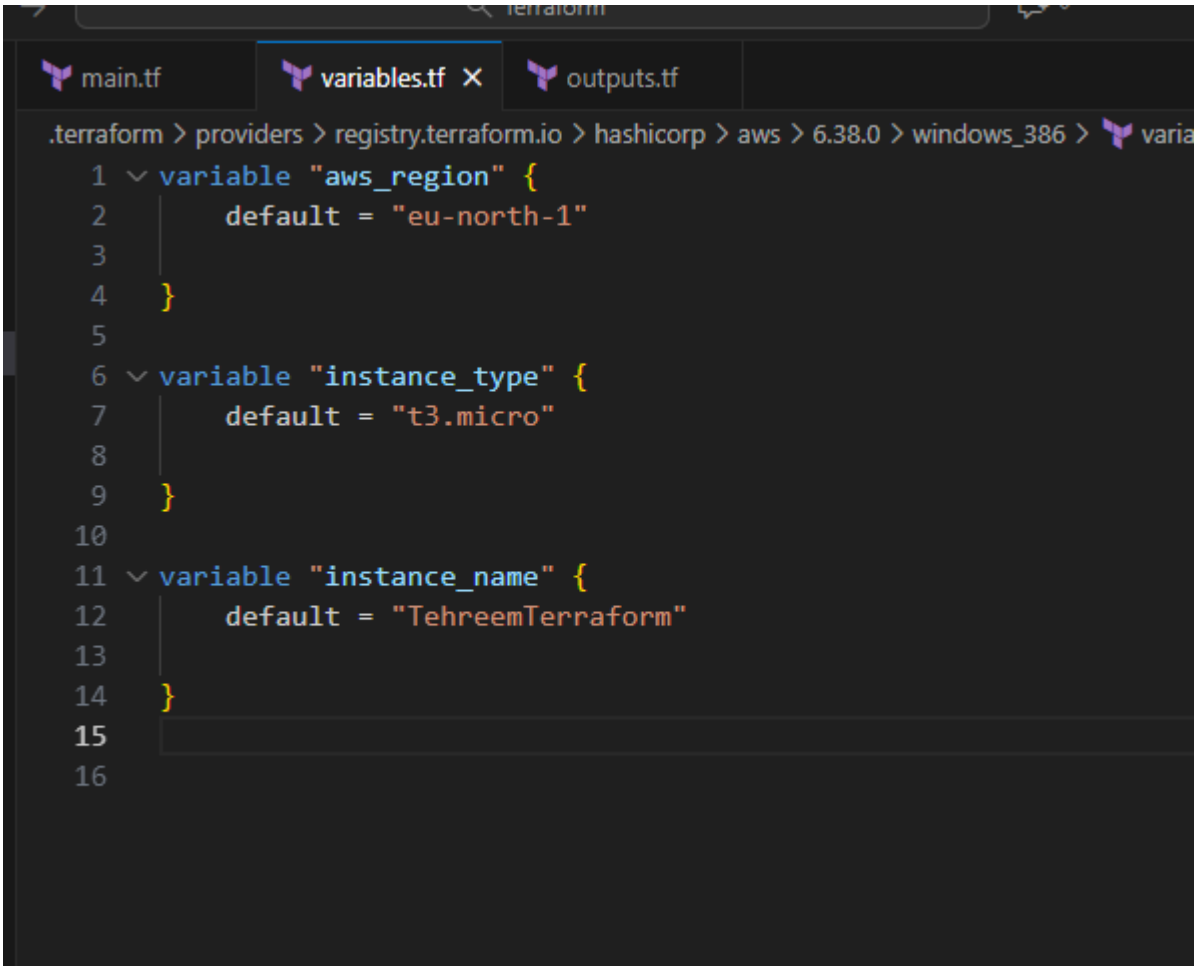
First instance has been created



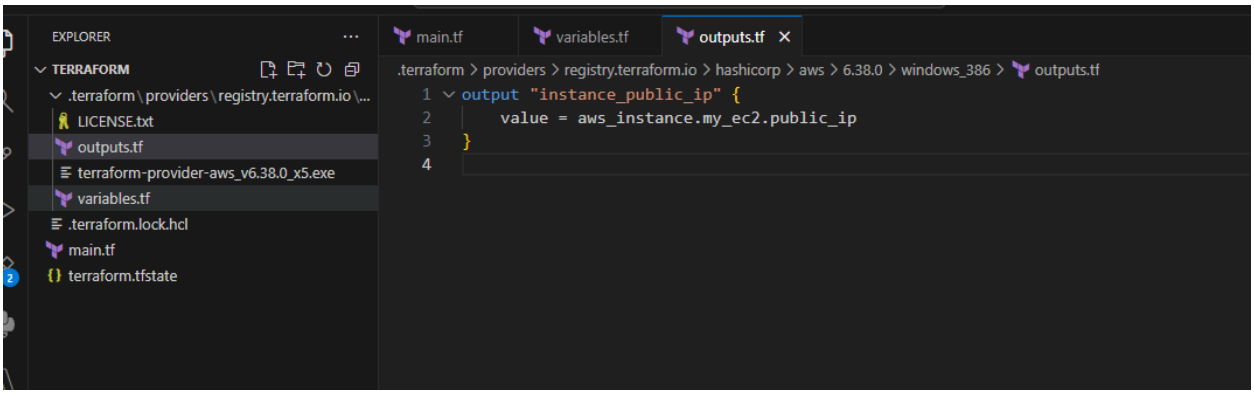


Terraform variables





If we don't want to check manually to aws that our instance is created or not we write output.tf file



Second instance has been created

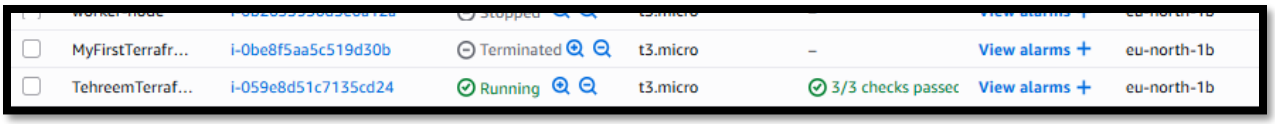
```
aws_instance.my_ec2: Creating...
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 15s [id=i-059e8d51c7135cd24]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

instance_public_ip = "16.171.27.9"
PS C:\Users\ASIF\Desktop\Terraform>
```

Activate Windows
Go to Settings to activate Wind



Conditional expression and locals

Main.tf file

```
# conditional expression & locals
provider "aws" {
    region =  var.aws_region
}

data  "aws_ami"   "amazon_linux"  {
    most_recent = true
    owners = ["amazon"]

    filter {
        name = "name"
        values = ["amzn2-ami-hvm-*-x86_64-gp2"]

    }
}
locals {
  name_tag = var.instance_type == "t3.micro" ? "Micro Instance" : "Standard Instan
}

resource "aws_instance" "my_ec2" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  tags = {
    Name = local.name_tag
  }
```

Third instance has been created

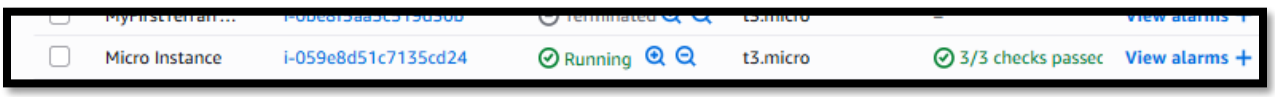
```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

Outputs:


Outputs:
Outputs:

instance_name_tag = "Micro Instance"
PS C:\Users\ASIF\Desktop\Terraform>
```



Depends on with:

Main.tf code:

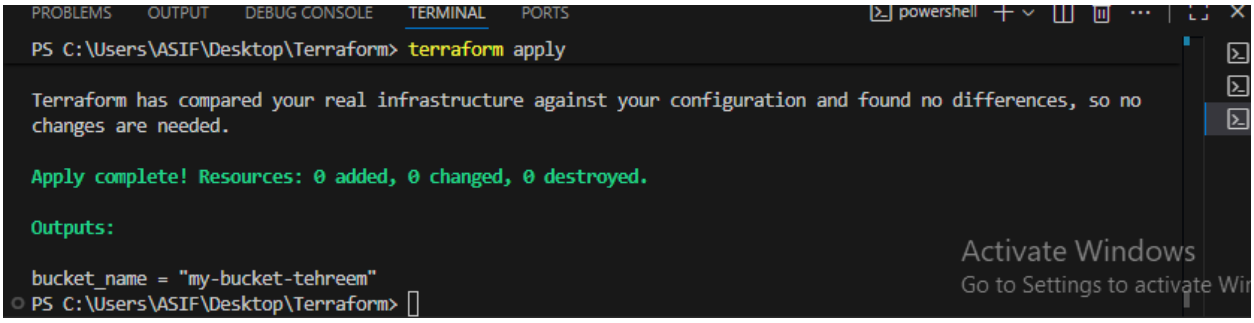


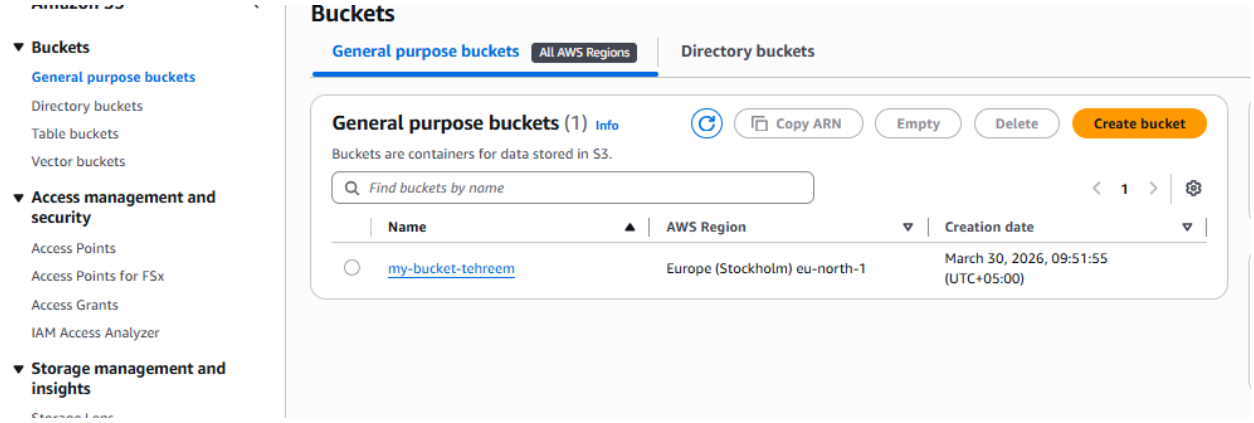
```
resource "aws_instance" "my_ec2" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  tags = {
    Name = var.instance_name
  }

  depends_on = [
    aws_s3_bucket.my_bucket
  ]
}
```

Bucket has been created:



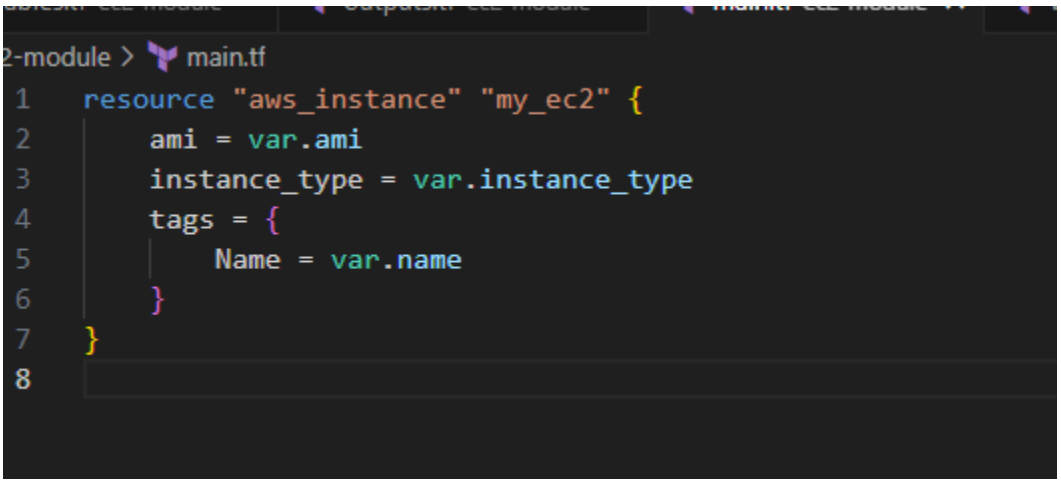


Local vs Remote Modules

Created an ec2-modules folder and in it made some file main.tf, variable.tf and outputs.tf



main.tf code



Outputs.tf code

ec2-module > outputs.tf

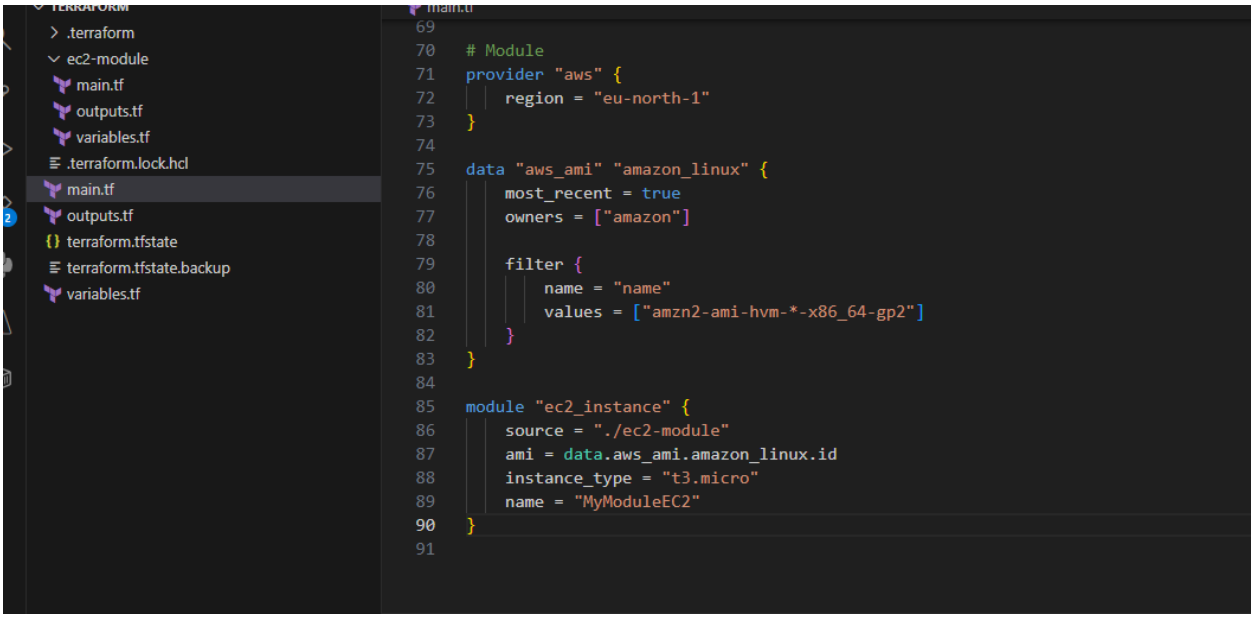
```
output "instance_id" {
  value = aws_instance.my_ec2.id
}
```

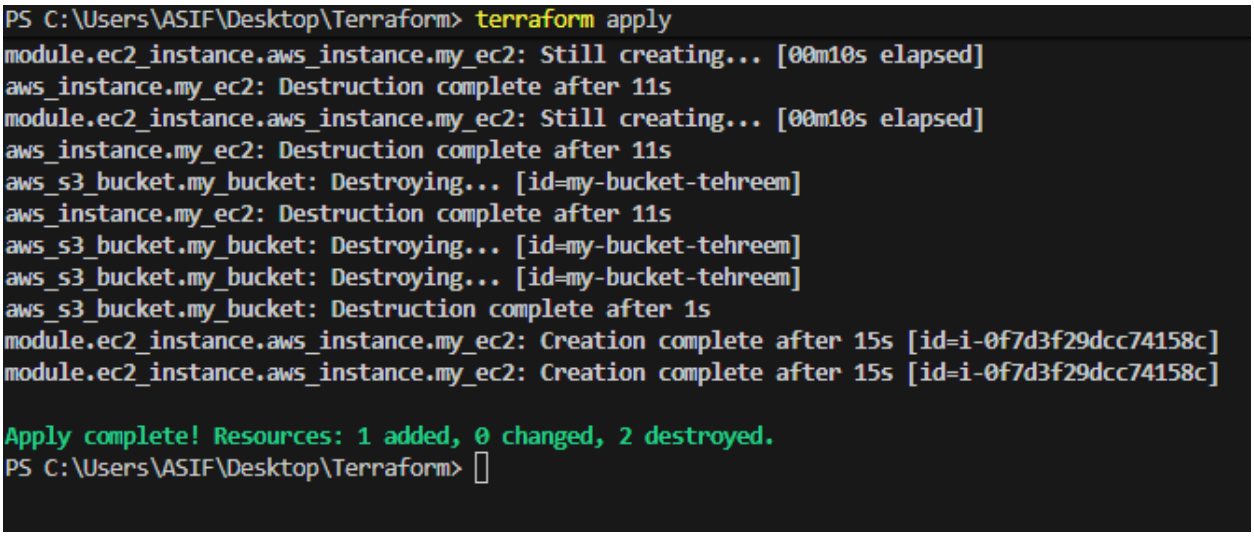
Variables.tf code

ec2-module > variables.tf

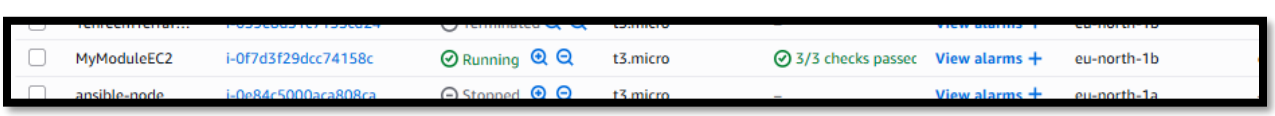
```
variable "ami" {}
variable "instance_type" {}
variable "name" {}
```

Main.tf file root folder





EC2 instance created using a module



Workspace

main.tf code root folder one

main.tf

```
92
93
94  # Workspaces
95
96  provider "aws" {
97      region = var.aws_region
98  }
99
100 data "aws_ami" "amazon_linux" {
101     most_recent = true
102     owners = ["amazon"]
103
104     filter {
105         name = "name"
106         values = ["amzn2-ami-hvm-*-x86_64-gp2"]
107     }
108 }
109
110 resource "aws_instance" "my_ec2" {
111     ami = data.aws_ami.amazon_linux.id
112     instance_type = var.instance_type
113
114     tags = {
115         Name = "EC2-${terraform.workspace}"
116         Environment = terraform.workspace
117     }
118 }
```

Outputs.tf file

```
  # Workspaces
output "workspace_name" {
  value = terraform.workspace
}
```

Workspace created

```
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 15s [id=i-015cc57481c65db2b]
aws_instance.my_ec2: Creation complete after 15s [id=i-015cc57481c65db2b]
module.ec2_instance.aws_instance.my_ec2: Still destroying... [id=i-0f7d3f29dcc74158c, 00m20s elapsed]
module.ec2_instance.aws_instance.my_ec2: Destruction complete after 20s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

workspace_name = "default"
```

Workspace instance been created

```
PS C:\Users\ASIF\Desktop\Terraform> terraform workspace list
* default

PS C:\Users\ASIF\Desktop\Terraform> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ASIF\Desktop\Terraform> terraform apply
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-037688ecd92e8611e]

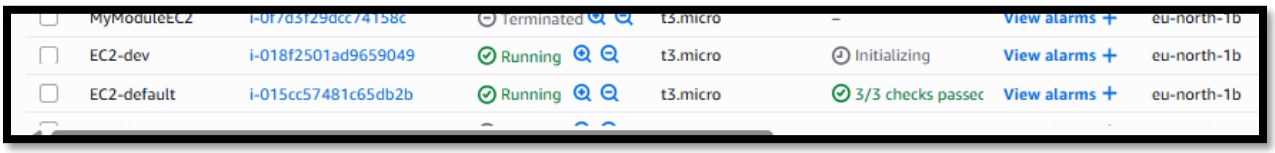
Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create
```

```
aws_instance.my_ec2: Creating...
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 15s [id=i-018f2501ad9659049]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

workspace_name = "dev"
PS C:\Users\ASIF\Desktop\Terraform>
```



Created a new workspace and doing same thing as above

```
workspace_name = "dev"
PS C:\Users\ASIF\Desktop\Terraform> terraform workspace new stage
Created and switched to workspace "stage"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ASIF\Desktop\Terraform> terraform apply
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-037688ecd92e8611e]

Terraform used the selected providers to generate the following execution plan. Resource action
indicated with the following symbols:
  + create

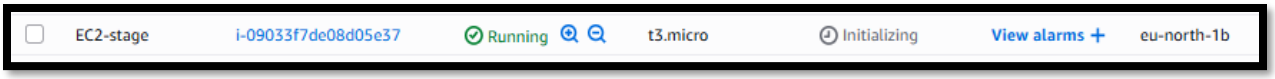
Terraform will perform the following actions:
```

```
aws_instance.my_ec2: Creating...
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 15s [id=i-09033f7de08d05e37]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

workspace_name = "stage"
PS C:\Users\ASIF\Desktop\Terraform>
                                                    Ln 19, Col 2    Space
```



Terraform Registry Modules

Main.tf file code

```hcl
# Terraform Registry Modules
provider "aws" {
    region = var.aws_region
}

module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["eu-north-1a", "eu-north-1b", "eu-north-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = true

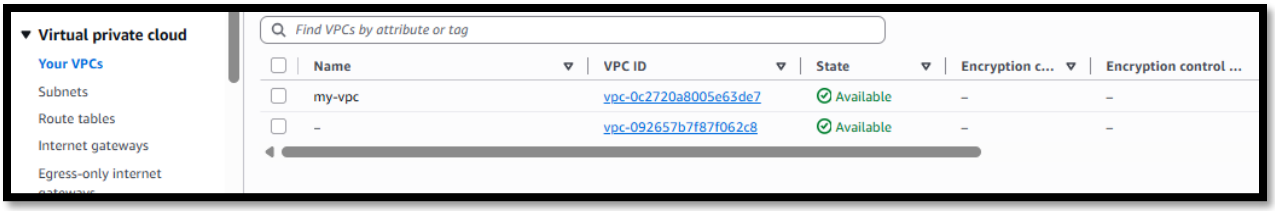
  tags = {
    Terraform = "true"
    Environment = "dev"
  }
}

```

VPC has been created

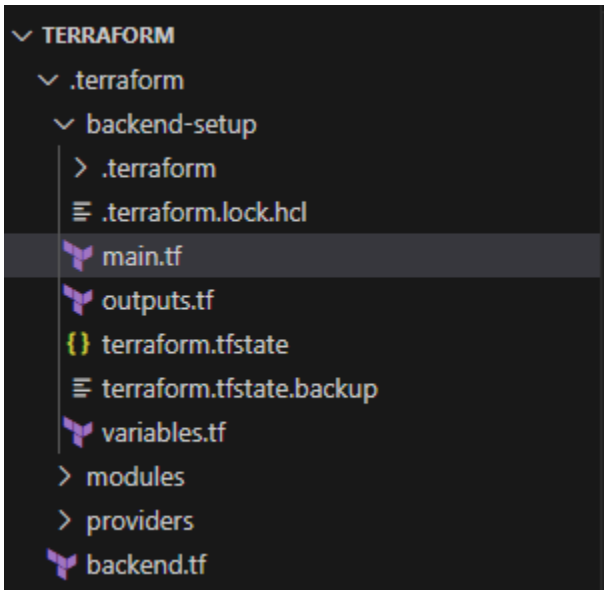
```
PS C:\Users\ASIF\Desktop\Terraform> terraform apply
module.vpc.aws_nat_gateway.this[2]: Creation complete after 1m37s [id=nat-0974089f12dfc7738]
module.vpc.aws_nat_gateway.this[0]: Still creating... [01m40s elapsed]
module.vpc.aws_nat_gateway.this[0]: Creation complete after 1m47s [id=nat-031c13f6dcc31c62e]
module.vpc.aws_route.private_nat_gateway[1]: Creating...
module.vpc.aws_route.private_nat_gateway[2]: Creating...
module.vpc.aws_route.private_nat_gateway[0]: Creating...
module.vpc.aws_route.private_nat_gateway[1]: Creation complete after 1s [id=r-rtb-08860ff26eb3324341080289494]
module.vpc.aws_route.private_nat_gateway[2]: Creation complete after 1s [id=r-rtb-02b3fc7ef0387ab381080289494]
module.vpc.aws_route.private_nat_gateway[0]: Creation complete after 1s [id=r-rtb-05de45383ed410f591080289494]

Apply complete! Resources: 32 added, 0 changed, 1 destroyed.
```



Remote State with S3 & Locking using DynamoDB

Created a backend-setup folder in that created some files main.tf, variables.tf and outputs.tf



Main.tf code

```hcl
provider "aws" {
  region = var.region
}

# S3 Bucket for Terraform State
resource "aws_s3_bucket" "terraform_state" {
  bucket        = var.bucket_name
  force_destroy = true

  tags = {
    Name        = "Terraform State Bucket"
    Environment = var.environment
  }
}

# Enable Versioning
resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Enable Encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
```

Activate
Go to Settin


iform > backend-setup > main.tf
resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
rule {
apply_server_side_encryption_by_default {
sse_algorithm = "AES256"
}
}
}
DynamoDB Table for Locking
resource "aws_dynamodb_table" "terraform_locks" {
name = var.lock_table
billing_mode = "PAY_PER_REQUEST"
hash_key = "LockID"
attribute {
name = "LockID"
type = "S"
}
tags = {
Name = "Terraform Lock Table"
Environment = var.environment
}
}

```
iform > backend-setup > main.tf
  resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "AES256"
      }
    }
  }

  # DynamoDB Table for Locking
  resource "aws_dynamodb_table" "terraform_locks" {
    name         = var.lock_table
    billing_mode = "PAY_PER_REQUEST"
    hash_key     = "LockID"

    attribute {
      name = "LockID"
      type = "S"
    }

    tags = {
      Name        = "Terraform Lock Table"
      Environment = var.environment
    }
  }
```

Outputs.tf code

```
output "s3_bucket_name" {
  value = aws_s3_bucket.terraform_state.bucket
}
output "dynamodb_table_name" {
  value = aws_dynamodb_table.terraform_locks.name
}
```

Variables.tf code

.terraform > backend-setup > variables.tf

```
1   variable "region" {
2       default = "eu-north-1"
3   }
4   variable "bucket_name" {
5     default = "tehreem-state-123456789"
6   }
7   variable "lock_table" {
8       default = "terraform-locks"
9   }
10  variable "environment" {
11      default = "dev"
12  }
```

```
PS C:\Users\ASIF\Desktop\Terraform\.terraform\backend-setup> terraform apply

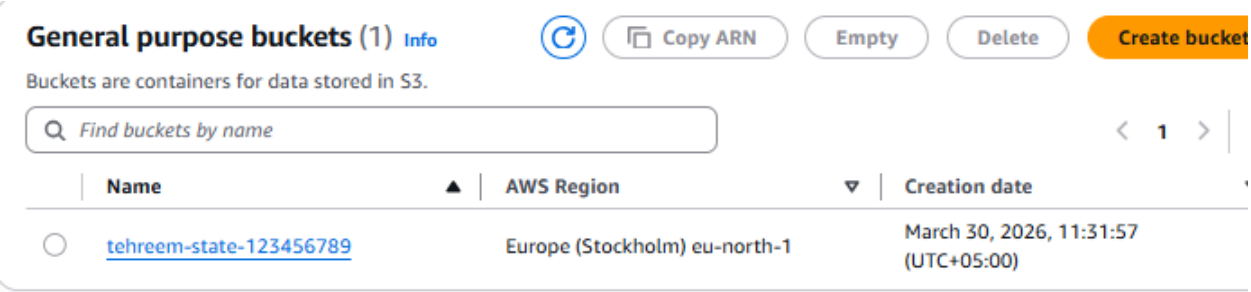
aws_s3_bucket.terraform_state: Creating...
aws_s3_bucket.terraform_state: Creation complete after 5s [id=tehreem-state-123456789]
aws_s3_bucket_versioning.versioning: Creating...
aws_s3_bucket_server_side_encryption_configuration.encryption: Creating...
aws_s3_bucket_server_side_encryption_configuration.encryption: Creation complete after 1s [i
123456789]
aws_s3_bucket_versioning.versioning: Creation complete after 2s [id=tehreem-state-123456789]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

dynamodb_table_name = "terraform-locks"
s3_bucket_name = "tehreem-state-123456789"
PS C:\Users\ASIF\Desktop\Terraform\.terraform\backend-setup>
```

Creation of S3 Bucket for Terraform State Storage



Creation of DynamoDB Table for State Locking

